

## **Faculty of Computing**

### **CS110: Fundamentals of Computer Programming**

**Class: BSAI 2026A**

## **Lab 02: Basic C++ Program Structure, Variables, I/O, and Debugging**

|              |                                                                                            |
|--------------|--------------------------------------------------------------------------------------------|
| <b>CLO 1</b> | Understand the syntax and semantics of different programming constructs                    |
| <b>CLO 3</b> | Build a program and associated documentation using appropriate IDE and supplementary tools |

**Time: \_2:00 pm – 4:50 pm\_**

**Instructor: Sahar Arshad**  
**Lab Engineer: Urooj Akmal**



### Introduction:

This lab builds directly on Lab 01, where students installed the C++ development environment and ran a basic welcome program. The focus now shifts from simply running code to understanding the structure of a small C++ program, choosing meaningful names and basic data types, using simple input/output, and learning how compiler messages help us debug mistakes. The activities are deliberately guided because this is still an introductory stage.

### Learning Objectives:

After completing this lab, you will be able to:

- Identify the role of `#include <iostream>`, `int main()`, braces, statements, semicolons, and `return 0` in a basic C++ program.
- Recognize common C++ tokens, including keywords, identifiers, literals, operators, and special symbols.
- Select suitable basic data types and use constants, comments, `std::cin`, `std::cout`, and simple escape sequences.
- Compile, run, test, and debug small C++ programs in the installed IDE and document the results clearly.

### Tools/Software Requirement:

- The C++ IDE installed and configured in Lab 01 (for example, Microsoft Visual Studio or the IDE specified by your instructor).
- A working C++ compiler integrated with the IDE.
- MS Word or equivalent for preparing the lab report.
- An online C++ compiler may be used only as an instructor-directed fallback or quick comparison; the main work should be demonstrated in the installed IDE.

### 03-Hour Lab Plan:

Briefing and IDE check: 10 minutes | Tasks 1-5: 150 minutes | Report completion and submission check: 20 minutes.

#### Task 1 [CLO 1] - Program Anatomy and Formatted Output (20 minutes):

Create a new C++ console program in your IDE and type the following starter program. Do not paste a previously completed Lab 01 file; use this task to become familiar with the structure again.



```
#include <iostream>

int main() {
    std::cout << "CS110 Programming Lab\n";
    std::cout << "Student:\tYourName\n";
    std::cout << "Section:\tYourSection\n";
    return 0;
}
```

1. Run the program once exactly as shown, then replace the sample name and section with your own information.
2. Add short comments in the code to identify the purpose of `#include <iostream>`, `int main()`, the braces `{ }`, one `std::cout` statement, and `return 0`. comments are added in the program by using `//`
3. Keep both `\n` and `\t` in the output and observe how they change the console layout.
4. In your report, write one short sentence explaining the purpose of the semicolon at the end of a statement.

**Deliverable:** C++ code, one output screenshot, and the short explanation asked above.

Task 2 [CLO 1] - Student Profile Generator: Variables, Data Types, and I/O (30 minutes):

Develop a small program that asks for student information and prints a neatly formatted profile. Use meaningful identifiers rather than single-letter names.

1. Declare `firstName` as `std::string`, `semester` as `int`, `cgpa` as `double`, `section` as `char`, and `isEnrolled` as `bool`. Set `isEnrolled` to `true` in the program.
2. Declare a named constant: `const std::string courseCode = "CS110";`. Include `#include <string>`.
3. Use `std::cin` to read `firstName`, `semester`, `cgpa`, and `section`. For this lab, enter only a one-word first name so that `std::cin >> firstName` is sufficient.
4. Use `std::cout` to display a profile containing the name, semester, CGPA, section, enrollment status, and course code. Use at least one `\t` or `\n` escape sequence for formatting.
5. Add a short comment beside each declaration explaining why that data type is appropriate.
6. Test the program

Suggested output layout (your values will be different):



----- CS110 STUDENT PROFILE -----

Name: ...  
Semester: ...  
CGPA: ...  
Section: ...  
Course: CS110

**Deliverable:** Code, screenshots of output, and the comments explaining your data-type choices.

Task 3 [CLO 1] - Token and Identifier (25 minutes):

This task focuses on reading small pieces of C++ before writing more code.

**Part A - Token check:** For the line `int totalMarks = 20;`, identify `int`, `totalMarks`, `=`, `20`, and `;` as a keyword, identifier, operator, literal, or special symbol.

**Part B - Repair the identifiers:** The declarations below contain invalid or poor identifiers. Rewrite them using valid, meaningful names and state the reason each original declaration is invalid.

```
int 2ndSemester = 2;  
double total marks = 87.5;  
char class = 'A';  
std::string first-name = "Ali";  
int return = 0;
```

1. Place your corrected declarations inside a complete C++ program and print each value using `std::cout`.
2. Add `const int maxLabSeats = 30;` to the corrected program. In your report, identify.
3. Compile and run the corrected program successfully.

**Deliverable:** Token classification, corrected declarations with reasons, final code, and one successful output screenshot.

Task 4 [CLO 3] - Compiler Detective: Find, Classify, and Fix Beginner Errors (40 minutes):

Work through the following three mini-debugging challenges. Read the first error message shown by the IDE, fix one issue at a time, and compile again. Record the important message or observation in your report rather than copying the entire error list.



### Challenge A

```
#include <iostream>

int Main() {
    std::cout << "Debugging starts here"
    return 0;
}
```

### Challenge B

```
#include <iostream>

int main() {
    char section = "A";
    int semester = 2
    std::cout << "Section: " << section << "\n";
    std::cout << "Semester: " << semestr << "\n";
    return 0;
}
```

### Challenge C

```
#include <iostream>

int main() {
    int first = 10, second = 20;
    double average = first + second / 2;
    int divisor;
    std::cout << "Enter a non-zero divisor: ";
    std::cin >> divisor;
    std::cout << "Average = " << average << "\n";
    std::cout << "10 / divisor = " << 10 / divisor << "\n";
    return 0;
}
```

1. For Challenge A, fix all problems and decide which issue is a syntax error and which issue can lead to a linker/build error.
2. For Challenge B, fix all three problems and explain what was wrong
3. For Challenge C, correct the average formula so that the result for 10 and 20 is 15. Test division only with a non-zero value. Explain why entering 0 would be unsafe at runtime. Condition-based protection against zero will be learned later in the course.
4. For each challenge, include the corrected code and a short note describing the error category: syntax/compile-time, linker/build, runtime danger, or logic error.



**Deliverable:** Corrected code for A-C, at least one screenshot showing an IDE error/build message, one screenshot of successful execution, and your error classifications.

Task 5 [CLO 3] - Build from a Specification: Mini Calculator (35 minutes):

Create this program from a blank source file. The purpose is to combine the complete beginner workflow: think in steps, write code, compile, run, test, and improve.

1. Before coding, write 6-8 lines of pseudocode using START, INPUT, PROCESS/CALCULATE, OUTPUT, and END to describe the solution.
2. Declare two variables of type double named firstNumber and secondNumber and read both values using `std::cin`.
3. Calculate and display the sum, difference, and product. Show each result on a separate line.
4. Use clear prompts, meaningful variable names, correct indentation, and at least two useful comments.
5. Test the program with (8, 3) and with at least one decimal test case, such as (7.5, 2.0).
6. At the end of your report, describe your own program in the four-step journey: source code -> compile -> run -> output.

**Deliverable:** C++ code, screenshots of both test runs, and the one-sentence source/compile/run/output explanation.

Deliverables:

Compile a single Word document and submit it on LMS. Organize the report under Task 1 through Task 5. For each task, include the code, output/IDE screenshot(s), and short explanation or observation. The submitted code must be the code demonstrated in the lab.

### Lab Rubrics

Your Lab 2 will be graded according to the following rubrics.

| Lab Rubrics for Lab 2 |                                |                                         |                                                       |                                                |                                              |
|-----------------------|--------------------------------|-----------------------------------------|-------------------------------------------------------|------------------------------------------------|----------------------------------------------|
| Sr.<br>No.            | Assessment                     | Unacceptable (0 Marks)                  | Does Not Meet Expectations (1/2 Marks)                | Meets Expectations (3/4 Marks)                 | Exceeds Expectations (5 Marks)               |
| 1                     | Basic C++ syntax and semantics | The student did not submit the required | The student is unable to identify the basic structure | The student demonstrates partial understanding | The student demonstrates clear understanding |



|   |                                                                |                                                                                       |                                                                                                                                                                                                       |                                                                                                                                                                                               |                                                                                                                                                                                                          |
|---|----------------------------------------------------------------|---------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | (CLO1, PLO1)                                                   | work, submitted work that cannot be evaluated, or used unfair means/plagiarized work. | of a C++ program or use variables/input/output correctly. Code contains major syntax errors and cannot be compiled without substantial help.                                                          | ng of program structure, tokens, identifiers, data types, and I/O. Most tasks are attempted and code can be made to run with limited guidance, although some syntax/semantic mistakes remain. | ng of the basic C++ structure, identifiers, literals/constants, data types, and I/O. Code is correct, readable, independently written, and explanations accurately connect syntax with program behavior. |
| 2 | IDE usage, compiling/debugging, and documentation (CLO3, PLO5) |                                                                                       | The student has difficulty creating/building/running a project in the IDE, cannot use compiler messages to correct common errors, and provides incomplete or poorly organized evidence/documentation. | The student can create, build, and run programs in the IDE and can fix common beginner errors with limited help. Code naming/comments and screenshots/documentation are mostly complete       | The student confidently uses the IDE to compile, run, test, and debug programs; interprets beginner compiler/build messages; uses clear names/comments; and submits complete, well-structured code,      |



|  |  |  |  |                   |                                          |
|--|--|--|--|-------------------|------------------------------------------|
|  |  |  |  | and<br>organized. | screenshots,<br>and<br>observation<br>s. |
|--|--|--|--|-------------------|------------------------------------------|

# Lab Task

## Task1

```
02_a.cpp U X
pf > LabTask02 > 02-a > 02_a.cpp > main()
2  using namespace std; //used this to make code more concise,
3  // makes it so that we dont have to specify that the current line uses the standard library each time
4
5  int main() { //the main function/entry point
6      cout << "CS110 Programming Lab\n"; //cout statement (console output)
7      cout << "Student:\t Abdur Rehman Tariq\n"; //semi colons are used to specify the end of a line (for the compiler)
8      cout << "Section:\t BS AI-A \n";
9      return 0; // since main is type int, returning 0 is makes it a flag to tell the compiler the what the end of the program is
10 } //closing braces to specify the end of a main function

Problems Output Debug Console Terminal Ports Spell Checker 3
❖ tariqmahmood@Tariqs-MacBook-Air 02-a % 02_a
zsh: command not found: 02_a
❖ tariqmahmood@Tariqs-MacBook-Air 02-a % ./02_a
CS110 Programming Lab
Student:      Abdur Rehman Tariq
Section:      BS AI-A
❖ tariqmahmood@Tariqs-MacBook-Air 02-a %
```

/t is the tab character,  
see how the string in the program doesn't have the  
amount of whitespace that the output does? That's what  
the '\t' character does.  
Meanwhile, the '\n' character is used to push console  
output to a new line instead of all on just one line



semi colons are used to specify the end of a line (for the compiler)

## Task2

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string firstName; //multiple characters so a string
6      int semesterNumber; //always an integer, never a decimal number so an int
7      char section; // always a single character never multiple so a char
8      double cgpa; // can be a decimal so a double not an int
9
10     std::cout << "Enter FirstName:\t";
11     std::cin >> firstName;
12     std::cout << "Enter Semester Number:\t";
13     std::cin >> semesterNumber;
14     std::cout << "Enter Section : \t";
15     std::cin >> section;
16     std::cout << "Enter Cgpa:\t";
17     std::cin >> cgpa;
18
19     std::cout << "\n----- CS110 STUDENT PROFILE ----- \n";
20     std::cout << "Name:\t" << firstName << "\n";
21     std::cout << "Semester:\t" << semesterNumber << "\n";
22     std::cout << "CGPA:\t" << cgpa << "\n";
23     std::cout << "Section:\t" << section << "\n";
24     std::cout << "Course:\tCS110\n";
25
26     return 0;
27 }
28
```

Problems Output Debug Console Terminal Ports Spell Checker 9

```
tariqmahmood@Tariqs-MacBook-Air 02-B % clang++ 02_B.cpp -o 02_b
tariqmahmood@Tariqs-MacBook-Air 02-B % ./02_B
Enter FirstName:      AbdurRehman
Enter Semester Number: ^C
tariqmahmood@Tariqs-MacBook-Air 02-B % clang++ 02_B.cpp -o 02_b
tariqmahmood@Tariqs-MacBook-Air 02-B % ./02_B
Enter FirstName:      AbdurRehman
Enter Semester Number: 1
Enter Section : A
Enter Cgpa:          3.3

----- CS110 STUDENT PROFILE -----
Name:   AbdurRehman
Semester: 1
CGPA:   3.3
Section: A
Course: CS110
tariqmahmood@Tariqs-MacBook-Air 02-B %
```



## Task3

### A) Token Check:

In The Line “`int totalMarks = 20;`” ‘`int`’ is a keyword, ‘`totalMarks`’ is the identifier, ‘`=`’ is the operator, ‘`20`’ is the literal and ‘`;`’ is the special symbol.

### B) `int 2ndSemester = 2;` ← starts with a number,

Correction: `int SeacondSemester = 2;`

`double total marks = 87.5;` ← has whitespace in the name

Correction: `double totalMarks = 87.5;`

`char class = 'A';` ← lowercase

Correction: `char Class = 'A'`

`std::string first-name = "Ali";` ← has a hyphen instead of being snakecase

Correction: `std::string firstName = "Ali";`

`int return = 0;` ← uses a reserved keyword

Correction: `int Return = 0;`



```
← → Untitled (Workspace)
02_a-2.png U × 02_a.cpp U × 02_b.cpp U × 02_b.png U × 02_c.cpp U ×
pf > LabTask02 > 02_C > 02_c.cpp > ...
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      int SeacondSemester = 2;
6      double totalMarks = 87.5;
7      char Class = 'A';
8      std::string firstName = "Ali";
9      int Return = 0;
10     const int maxLabSeats = 30;
11
12     std::cout << "SeacondSemester: " << SeacondSemester << "\n";
13
14     std::cout << "totalMarks: " << totalMarks << "\n";
15     std::cout << "Class: " << Class << "\n";
16
17     std::cout << "firstName: " << firstName << "\n";
18
19     std::cout << "Return: " << Return << "\n";
20     std::cout << "maxLabSeats: " << maxLabSeats << "\n";
21
22     return 0;
23 }
24

Problems Output Debug Console Terminal Ports Spell Checker 12
tariqmahmood@Tariqs-MacBook-Air 02_C % clang++ 02_c.cpp -o 02_c
tariqmahmood@Tariqs-MacBook-Air 02_C % ./02_c
SeacondSemester: 2
totalMarks: 87.5
Class: A
firstName: Ali
Return: 0
maxLabSeats: 30
tariqmahmood@Tariqs-MacBook-Air 02_C %
```

Here (line10) , “maxLabSeats” is the identifier  
and “30” is a literal



# Task4

## Challenge A:

```
pf > LabTask02 > 02_D > 02_d.cpp > Main()
1  #include <iostream>
2
3  int Main() {
4      std::cout << "Debugging starts here"
5      return 0;    expected a ';'
6  }
```

Problems 1 Output Debug Console Terminal Ports Spell Checker

```
• tariqmahmood@Tariqs-MacBook-Air LabTask02 % cd 02_D
• tariqmahmood@Tariqs-MacBook-Air 02_D % clang++ 02_d.cpp -o 02_d
02_d.cpp:4:41: error: expected ';' after expression
    4 |         std::cout << "Debugging starts here"
      |                                         ^
      |                                         ;
1 error generated.
❖ tariqmahmood@Tariqs-MacBook-Air 02_D %
```



Since its missing a SemiColon on line 4 and the m in “Main” is capital , adding a semicolon and making m lowercase fixes the issue. This is a Syntax error

```
G+ 02_d.cpp U X
pf > LabTask02 > 02_D > G+ 02_d.cpp > ...
1  #include <iostream>
2
3  int main() {
4      std::cout << "Debugging starts here";
5      return 0;
6  }
```

Problems Output Debug Console Terminal Ports Spell Checker

```
tariqmahmood@Tariqs-MacBook-Air 02_D % ./02_d
Debugging starts here
❖ tariqmahmood@Tariqs-MacBook-Air 02_D %
```

## Challenge B:

```
pf > LabTask02 > 02_D > G+ 02_d.cpp > main()
3  int main() {
4      char section = "A";      a value of type "const char *" cannot be used to initialize an entity of type "char"
5      int semester = 2;
6      std::cout << "Section: " << section << "\n";      expected a ';'
7      std::cout << "Semester: " << semestr << "\n";      identifier "semestr" is undefined
8      return 0;
9  }
```

Problems 3 Output Debug Console Terminal Ports Spell Checker 1

```
tariqmahmood@Tariqs-MacBook-Air 02_D % ./02_d
Debugging starts here
tariqmahmood@Tariqs-MacBook-Air 02_D % clang++ 02_d.cpp -o 02_d
02_d.cpp:4:10: error: cannot initialize a variable of type 'char' with an lvalue of type 'const char[2]'
4      | char section = "A";
      | ^~~~~
02_d.cpp:5:21: error: expected ';' at end of declaration
5      | int semester = 2;
      | ^
02_d.cpp:7:34: error: use of undeclared identifier 'semestr'; did you mean 'semester'?
7      | std::cout << "Semester: " << semestr << "\n";
      |                                ^~~~~~
      |                                semester
02_d.cpp:5:9: note: 'semester' declared here
5      | int semester = 2;
      | ^
3 errors generated.
tariqmahmood@Tariqs-MacBook-Air 02_D %
```



- Char uses double quotes instead of single quotes (line 4)
- No semicolon on line5
- 'e' missing in identifier semester
- these are all Compile time errors

After Fixing:

```
pf > LabTask02 > 02_D > 02_d.cpp > main()
1  #include <iostream>
2
3  int main() {
4      char section = 'A';
5      int semester = 2;
6      std::cout << "Section: " << section << "\n";
7      std::cout << "Semester: " << semester << "\n";
8      return 0;
9  }
```

Problems Output Debug Console Terminal Ports Spell Checker

```
tariqmahmood@Tariqs-MacBook-Air 02_D % clang++ 02_d.cpp -o 02_d
tariqmahmood@Tariqs-MacBook-Air 02_D % ./02_d
Section: A
Semester: 2
tariqmahmood@Tariqs-MacBook-Air 02_D %
```



## Challenge C:

```
02_d.cpp U x 02_d-1.png U x
pf > LabTask02 > 02_D > 02_d.cpp > main()
1  #include <iostream>
2
3  int main() {
4      int first = 10, second = 20;
5      double average = first + second / 2;
6      int divisor;
7      std::cout << "Enter a non-zero divisor: ";
8      std::cin >> divisor;
9      std::cout << "Average = " << average << "\n";
10     std::cout << "10 / divisor = " << 10 / divisor << "\n";
11     return 0;
12 }
```

Problems Output Debug Console Terminal Ports Spell Checker

```
● tariqmahmood@Tariqs-MacBook-Air 02_D % clang++ 02_d.cpp -o 02_d
● tariqmahmood@Tariqs-MacBook-Air 02_D % ./02_d
Section: A
Semester: 2
● tariqmahmood@Tariqs-MacBook-Air 02_D % clang++ 02_d.cpp -o 02_d
● tariqmahmood@Tariqs-MacBook-Air 02_D % ./02_d
Enter a non-zero divisor: 2
Average = 20
10 / divisor = 5
tariqmahmood@Tariqs-MacBook-Air 02_D %
```

While there may not be any syntax/semantic errors, there are Logical error(s) here, particularly on line 5 which should be `:double average = (first + second) / 2`, not `"first + second / 2"` which is read by the compiler as `"first + (second / 2)"` ←incorrect. Hence, this is a logical error



```
02_d.cpp U × 02_d-1.png U ×
pf > LabTask02 > 02_D > 02_d.cpp > main()
1  #include <iostream>
2
3  int main() {
4      int first = 10, second = 20;
5      double average = (first + second) / 2;
6      int divisor;
7      std::cout << "Enter a non-zero divisor: ";
8      std::cin >> divisor;
9      std::cout << "Average = " << average << "\n";
10     std::cout << "10 / divisor = " << 10 / divisor << "\n";
11     return 0;
12 }
```

Problems Output Debug Console Terminal Ports Spell Checker

```
• tariqmahmood@Tariqs-MacBook-Air 02_D % clang++ 02_d.cpp -o 02_d
• tariqmahmood@Tariqs-MacBook-Air 02_D % ./02_d
Enter a non-zero divisor: 2
Average = 15
10 / divisor = 5
❖ tariqmahmood@Tariqs-MacBook-Air 02_D %
```

Corrected code above



## Task5

I first tried to think of using loops, adding validation, maybe even try to look for the c++ equivalent of the python eval( ) function. But after being instructed not to do any of that. I used the instructions in the lms to make a simple cout based calc which works for 2 inputs both declared on the first line. initially I made them both integers but when I tested the decimal number case, I changed the inputs to be of type double.. After that, to avoid having to assign the variable names or datatypes, I outputted the result of each operation directly on a new line instead of storing it in a variable. Also, I thought of using "\*" and "/" in the output to show the result but a non technical user may not understand those symbols so I instead decided on using conventional maths symbols for multiplication and division. Finally, I put the \n character after each line to make it look less congested.

```
02_d.cpp U X 02_e.cpp U X
pf > LabTask02 > 02_E > 02_e.cpp > main()
1  #include <iostream>
2
3
4  int main () {
5      double FirstNumber, SecondNumber; //declaring before hand to make it more concise later
6      std::cout << "Enter the First Number: ";
7      std::cin >> FirstNumber;
8      std::cout << "Enter the Second Number: ";
9      std::cin >> SecondNumber;
10
11     // using math symbols and directly outputting the result of each calculation instead of storing it in a variable
12     std::cout << FirstNumber << " + " << SecondNumber << " = " << FirstNumber + SecondNumber << "\n";
13     std::cout << FirstNumber << " - " << SecondNumber << " = " << FirstNumber - SecondNumber << "\n";
14     std::cout << FirstNumber << " x " << SecondNumber << " = " << FirstNumber * SecondNumber << "\n";
15
16     std::cout << FirstNumber << " / " << SecondNumber << " = " << FirstNumber / SecondNumber << "\n";
17     return 0;
18 }
```

```
Problems Output Debug Console Terminal Ports Spell Checker
• tariqmahmood@Tariqs-MacBook-Air 02_E % clang++ 02_e.cpp -o 02_e
• tariqmahmood@Tariqs-MacBook-Air 02_E % ./02_e
Enter the First Number: 8
Enter the Second Number: 3
8 + 3 = 11
8 - 3 = 5
8 x 3 = 24
8 / 3 = 2.66667
• tariqmahmood@Tariqs-MacBook-Air 02_E % ./02_e
Enter the First Number: 7.5
Enter the Second Number: 2.0
7.5 + 2 = 9.5
7.5 - 2 = 5.5
7.5 x 2 = 15
7.5 / 2 = 3.75
• tariqmahmood@Tariqs-MacBook-Air 02_E %
```



**Github Repo For All Code/Files:**  
<https://github.com/ARTariqDev/Nust-AI-2k26>

**Abdur Rehman TARIQ,  
BS AI-A 2K26,  
CMS ID: 623486**